

AICL Editor

Web-Based Development Environment

User Manual

Version 2.0.0

Philippe-Antoine | June 2026

- AI Chat Panel with context-aware AICL assistant
- Agentic AI: create, compile, and verify files from chat
 - Structured error display with Explain Error button
- One-click compilation to Python, Rust, JavaScript, Go
- Proof of Origin viewer and independent verification
 - Interactive exercises and REPL

Table of Contents

1 Introduction

2 Interface Layout

- 2.1 Toolbar
- 2.2 Left Panel: Examples and Exercises
- 2.3 Center Panel: Code Editor
- 2.4 Right Panel: Output, Tree, Code, and Chat
- 2.5 Bottom Panel: Output, REPL, and Exercises
- 2.6 Status Bar

3 AI Chat Panel

- 3.1 Opening the Chat
- 3.2 Chat Capabilities
- 3.3 Agentic AI: The AICL_FILE Protocol
- 3.4 Action Buttons: Create, Compile, Verify
- 3.5 Offline Fallback

4 Error Display and Explanation

- 4.1 Error Box in Chat
- 4.2 Explain Error Button
- 4.3 Try Verify First Button
- 4.4 Automatic Error Propagation

5 Editor Features

6 Compilation and Verification

- 6.1 Compiling
- 6.2 Verifying
- 6.3 Auditing
- 6.4 Provenance Explanation

7 Examples

8 Exercises

9 Interactive REPL

10 Keyboard Shortcuts

11 Troubleshooting

1. Introduction

The AICL Web Editor is a browser-based development environment for the Architecture Compilation Language, inspired by Python's IDLE but designed specifically for AICL's unique workflow. Like IDLE provides an integrated editor, shell, and debugger for Python, the AICL Editor provides a specification editor, compilation engine, proof viewer, and interactive REPL for AICL. Version 2.0 introduces a major new capability: an integrated AI Chat panel powered by an LLM with deep AICL domain knowledge, including agentic capabilities that allow the AI to create files, compile code, and verify specifications directly from the conversation.

The editor features a dark theme, AICL syntax highlighting, multi-file tabs, and one-click access to all AICL compilation, verification, and audit features. The AI assistant can understand your current code, write new AICL specifications from natural language descriptions, and automatically compile and verify them. When errors occur, they are displayed with structured details and an Explain Error button that provides root-cause analysis with specific fix suggestions.

The editor is organized into five main areas: the toolbar at the top for actions, the left panel for examples and exercises, the center panel for code editing, the right panel for output, tree, generated code, and AI chat, and the bottom panel for output and REPL. An interactive REPL at the bottom provides a command-line interface similar to the TUI, allowing you to type commands and AICL code directly.

2. Interface Layout

2.1 Toolbar

The toolbar provides one-click access to all AICL operations, organized from left to right. Each button triggers an immediate action on the current file. The AI Chat button, new in v2.0, opens the integrated AI assistant panel.

Button	Description
New	Create a new untitled .aicl file in a new tab
Open	Open a .aicl file from your local filesystem
Save	Download the current file as .aicl
Target selector	Choose compilation target (Python, Rust, JavaScript, Go)
Compile	Compile the current AICL source to the selected target
Verify	Check specification completeness, coherence, and satisfaction
Audit	Run audit on compiled output (coverage, orphan detection)
Explain	Show provenance explanation for each compilation decision
Tree	Display the architecture tree of the current specification
Optimize	Suggest architectural optimizations

Button	Description
AI Chat	Open the AI assistant chat panel (v2.0)
Clear	Clear the output panel
Find	Search and replace within the editor

2.2 Left Panel: Examples and Exercises

The left panel has two tabs: Examples and Exercises. The Examples tab lists built-in AICL programs ranging from simple (Blue Square, Level 1) to complex (Banking, Level 10). Click any example to load it into the editor. The Exercises tab lists five progressive exercises with descriptions and starter templates. Click an exercise to load its template with TODO markers.

2.3 Center Panel: Code Editor

The code editor supports AICL syntax highlighting with keywords in cyan, types in green, and comments in gray. It includes line numbers, auto-indentation (4 spaces after colons), Tab/Shift+Tab for indent/outdent, and Ctrl+F for find and replace. Multiple files can be open simultaneously in tabs at the top of the editor. Each tab shows the filename and a close button. The current file is highlighted. When the AI creates a new file through the agentic protocol, it automatically appears as a new tab in the editor.

2.4 Right Panel: Output, Tree, Code, and Chat

The right panel has four tabs, one of which is new in v2.0:

Tab	Description
Output	Compilation results, verification reports, audit summaries with timestamps
Tree	Architecture tree visualization of the current specification
Code	Generated source code and test code with copy buttons
Chat	AI assistant conversation panel (v2.0)

The Chat tab provides a full conversational AI assistant that understands AICL, can read your current code, and can take agentic actions like creating files and compiling them. See Section 3 for the complete chat documentation.

2.5 Bottom Panel: Output, REPL, and Exercises

The bottom panel has three tabs: Output (duplicate of right panel output for convenience), REPL (an interactive command shell), and Exercises (exercise details and starter templates). The REPL supports colon-prefixed commands just like the TUI (:compile, :verify, :help, etc.) and can also accept AICL code directly.

2.6 Status Bar

The status bar at the very bottom shows: current filename, cursor position (line:column), selected target language, and AICL version number.

3. AI Chat Panel

The AI Chat panel is a full conversational AI assistant embedded directly in the editor. It uses a large language model with deep AICL domain knowledge to provide context-aware assistance. The chat is not a simple Q&A; bot; it is an agentic system that can understand your code, write new specifications, create files, compile them, and verify them, all through the conversation interface.

3.1 Opening the Chat

There are two ways to open the AI Chat panel. First, click the purple AI Chat button in the toolbar (marked with a message square icon). Second, click the Chat tab in the right panel. Both actions will switch the right panel to the Chat tab and auto-focus the input field. The chat input field has a placeholder suggesting: "Try 'Describe a todo app in AICL'..."

3.2 Chat Capabilities

The AI assistant has comprehensive knowledge of the AICL language and the editor itself. When you send a message, the current code in your editor is automatically included as context, so the AI can analyze and respond to the spec you are actively working on. The conversation is multi-turn, meaning the AI remembers your entire conversation history for the session.

The AI assistant can help with:

- Writing AICL specifications from natural language descriptions
- Explaining AICL concepts like the No-Orphan Property, Proof of Origin, and provenance types
- Debugging compilation errors with specific fix suggestions
- Suggesting improvements to specifications for better audit coverage
- Guiding you through the editor workflow (how to compile, verify, audit)
- Providing keyboard shortcuts and REPL commands
- Generating complete AICL programs that can be compiled and verified

When you ask the AI to write an AICL program, it uses the agentic `:::AICL_FILE` protocol (described below) to output code that you can immediately create, compile, and verify with one click.

3.3 Agentic AI: The `:::AICL_FILE` Protocol

When the AI generates AICL code, it does not output it in plain markdown code blocks. Instead, it uses a structured format called the `:::AICL_FILE` protocol. This protocol allows the editor to parse the AI's output, detect AICL files, and present them as interactive code cards with action buttons.

The format is:

```
:::AICL_FILE filename.aicl
(Goal Build ...)
(Layer ...)
...
:::END_FILE
```

The editor parses these blocks and renders each one as a card in the chat showing the filename, a preview of the code, and three action buttons. The AI is instructed to always use this format when writing AICL code, ensuring that every code snippet is immediately actionable. Multiple `:::AICL_FILE` blocks can appear in a single AI response, and explanatory text can appear before or after the blocks.

3.4 Action Buttons: Create, Compile, Verify

Each `:::AICL_FILE` code card renders with three action buttons that let you take immediate action on the AI-generated code without leaving the chat:

Button	Color	Action
Create File	Teal	Creates a new editor tab with the AI-generated code and switches to it
Create + Compile	Red	Creates the file AND immediately compiles it; reports results in chat
Verify	Blue	Creates the file AND immediately runs verification; reports results in chat

When you click Create + Compile, the editor creates the file, selects the current target language (Python, Rust, JavaScript, or Go), and compiles it. The result is reported back into the chat: on success, you see a message with the number of compilation stages completed, audit coverage percentage, and proof validity. On failure, a structured error box appears with details (see Section 4).

When you click Verify, the editor creates the file and runs the three-level verification check (completeness, coherence, satisfaction). On success, you see a confirmation that all checks passed. On failure, an error box appears with the specific check that failed and its details.

3.5 Offline Fallback

When the AI service is unavailable (network error, server down, or API failure), the chat API returns categorized offline responses based on keyword matching of your last message. The responses cover common topics like compilation workflow, the No-Orphan Property, Proof of Origin, exercises, and keyboard shortcuts. The response includes a 'connected: false' indicator so you know the AI is operating in offline mode. The editor remains fully functional even without the AI service.

4. Error Display and Explanation

Version 2.0 introduces a comprehensive error display system in the chat panel. When any operation fails (compile, verify, audit, or chat), the error is automatically displayed as a structured error box in the chat conversation. This is a significant improvement over the previous behavior where errors only appeared briefly in the output panel.

4.1 Error Box in Chat

The error box is rendered as a dark red card in the chat with the following elements: a red X-circle icon, the error message in bold, detailed error lines prefixed with arrow symbols, and action buttons at the bottom. Each error has an operation type (compile, verify, audit, or chat), a human-readable message, and an optional array of detailed error lines from the underlying system.

The ChatError interface that structures these errors:

```
interface ChatError {  
  operation: string; // 'compile', 'verify', 'audit', 'chat'  
  message: string; // Human-readable error summary  
  details?: string[]; // Array of detailed error lines  
}
```

The compile API enhances raw Python error messages with contextual fix suggestions before they reach the chat. For example, a 'missing Goal' error will include a 'Fix: Add a Goal section at the top' suggestion. This enhancement ensures that even before clicking Explain Error, you already have actionable guidance.

4.2 Explain Error Button

At the bottom of every error box is a red Explain Error button with a bug icon. Clicking it sends the full error context (operation type, message, and details) to the AI assistant, which then provides a detailed root-cause analysis. The AI follows a strict WHAT/WHY/FIX pattern:

- **WHAT** went wrong, explained in plain language
- **WHY** it happened, referencing AICL concepts and the relevant specification elements
- **FIX** with specific before/after code examples that you can apply immediately

The AI explanation references the relevant AICL keywords, language levels, and formal properties. It never just says 'fix the error' but always provides concrete, actionable steps. This turns compilation failures into learning opportunities.

4.3 Try Verify First Button

When the error comes from a failed compilation, an additional teal button appears: Try Verify First. This button creates the file and runs the verification check instead of compilation. The reasoning is that verification catches structural issues (missing Goal, incomplete Risk/Recovery pairs, dangling references) before compilation attempts code generation, making it easier to diagnose the root cause. Clicking this button is equivalent to the Verify action button on the code card.

4.4 Automatic Error Propagation

Errors are automatically pushed to the chat panel whenever the chat panel is open and an operation fails. This applies to all toolbar operations as well as agentic chat actions. Specifically, errors appear in chat when: compilation fails while the chat panel is open, verification shows FAIL or WARN while the chat is open, audit finds orphans while the chat is open, a network or server error occurs during any operation, or the chat API itself fails to connect.

This automatic propagation ensures that you never miss an error. Even if you triggered the compile from the toolbar rather than the chat, the error will still appear in the chat conversation where you can click Explain Error for a detailed analysis.

5. Editor Features

The AICL Editor provides a complete set of editing features comparable to Python's IDLE, adapted for the AICL language. These features support both manual editing and AI-assisted development.

Feature	Description	Shortcut
Syntax Highlighting	Keywords (cyan), Types (green), Comments (gray)	Always on
Auto-indentation	4 spaces after colons, extra indent for sub-sections	Automatic
Line Numbers	Displayed on the left side of the editor	Always visible
Indent/Outdent	Indent or outdent selected lines	Tab / Shift+Tab
Find and Replace	Search within the current file	Ctrl+F
Multiple File Tabs	Open and edit multiple files simultaneously	Click tabs
New File	Create a new untitled .aicl file	New button
Open File	Load a .aicl file from your computer	Open button
Save/Download	Download the current file as .aicl	Save button
Auto-complete	Suggests AICL keywords as you type	Tab
Copy	Copy compiled source or tests to clipboard	Copy button
Generated Code	View and copy generated source and test files	Code tab

6. Compilation and Verification

6.1 Compiling

Click the Compile button or use the REPL command `:compile` to compile the current AICL source. Select the target language from the dropdown (Python, Rust, JavaScript, Go) before compiling. The output panel shows the compilation result including audit coverage, TODO count, and proof validity. The Code tab displays the generated source and test files.

If compilation fails and the chat panel is open, a structured error box appears in the chat with details about the failure. The error box includes an Explain Error button and a Try Verify First button. See Section 4 for details.

6.2 Verifying

Click Verify to check specification quality before compiling. The verifier runs three levels of checks: Completeness (all required elements present), Coherence (no contradictions or dangling references), and Satisfaction (implementable and testable). Each check shows PASS, WARN, or FAIL status. Running Verify before Compile is recommended to catch structural issues early.

6.3 Auditing

Click Audit after compiling to see the audit report. The report shows total artifacts, artifacts with provenance, orphan count, and coverage percentage. The target is always 100% coverage with zero orphans (the No-Orphan Property). If orphans are found and the chat is open, an error box appears explaining the No-Orphan Property and which artifacts are orphaned.

6.4 Provenance Explanation

Click Explain to see why each line of generated code was produced. The explanation shows the provenance decision type, the source specification element, and the confidence level. All decisions are deterministic and fully auditable. AICL tracks 23 provenance types covering every compilation zone, from PATTERN_MATCH and SUB_LANGUAGE to AI_GENERATION and SPEC_EVOLVED.

7. Examples

Five built-in examples demonstrate progressively more advanced AICL features, from simple graphics to a complete banking system with AI self-writing:

Example	Levels	Description	Artifacts
Blue Square	1	Simple graphics application	35 artifacts, 100% audit
Pong Game	1-6	Game with entities, behaviors, conditions, events, parallelism	52 artifacts, 100% audit
Chat App	1-9	Full chat with security, optimization, learning	58 artifacts, 100% audit
Chess Game	1-9	Complex state management with network	59 artifacts, 100% audit
Banking System	1-10	Complete banking with native code	100% audit

8. Exercises

Five progressive exercises teach AICL from basics to advanced. Each loads a starter template with TODO markers that you complete. You can also ask the AI assistant for hints on any exercise by describing what you are stuck on in the chat panel.

#	Title	Description	What You Learn
1	Hello World	Create a minimal AICL program	Goal, Layer, Validation
2	Risk and Recovery	Add error handling to a file reader	Risk/Recovery pairs
3	Entities and Behaviors	Define data structures with actions	Entity, Behavior, Action
4	Conditions and Events	Add rules and event handlers	When/Then, On/Action
5	Full Application	Build a complete chat application	All 9 levels combined

To start an exercise, click the Exercises tab in the left panel, then click the exercise title. The starter template loads in the editor with TODO markers showing where to add code. Complete the TODOs and click Compile to test your solution. Click Verify to check specification quality. If you get stuck, open the AI Chat and ask for help with the specific exercise.

9. Interactive REPL

The REPL (Read-Eval-Print Loop) at the bottom of the editor provides an interactive command interface. Type colon-prefixed commands for actions, or type AICL code directly to add it to the current file. The REPL supports the same commands as the CLI TUI:

Command	Action
:help	Show all REPL commands
:compile	Compile current file
:verify	Verify specification
:audit	Audit compilation
:explain	Explain provenance
:tree	Architecture tree
:optimize	Optimize architecture
:clear	Clear output
:load	Load an example
:exercises	List exercises

10. Keyboard Shortcuts

Shortcut	Action
Ctrl+F	Find and Replace
Tab	Indent / Auto-complete
Shift+Tab	Outdent
Ctrl+S	Save/Download file
Enter (in chat)	Send message to AI assistant

11. Troubleshooting

Issue	Solution
Compilation errors	Use Verify before Compile to catch specification issues early. If errors appear, click Explain Error in the chat for detailed analysis.
Empty output	Make sure you clicked Compile first. Audit and Explain require a compilation result.
File not loading	Ensure the file has a .aicl extension and valid UTF-8 encoding.
REPL not responding	Click inside the REPL input field to focus it.
Copy button not working	Your browser may block clipboard access. Use Ctrl+C manually.
AI Chat not responding	Check that the AI service is available. The chat provides offline fallback responses when disconnected.
Error box in chat	Click Explain Error for root-cause analysis, or Try Verify First for structural checks before compilation.
Agentic actions failing	Ensure the code in the <code>:::AICL_FILE</code> block is valid AICL. Click Verify first to check structural completeness.